Software Testing – 6 – Testing Strategies

Lecture # 28

1

Recap

- In the two last lectures, we had discussed black-box and white-box testing techniques in quite a bit of detail
- We had also discussed certain techniques within this these two broad testing approaches

Today's Lecture

- Today we'll talk about testing strategies

- Software testing needs to be planned in advance and conducted systematically
- A number of software testing strategies have been proposed in the literature
- All of them have these four generic characteristics

Characteristics of Testing Strategies - 1

- (1) Testing begins at the component level and works “outward” toward the integration of the entire computer-based system
- (2) Different testing techniques are appropriate at different points in time

Characteristics of Testing Strategies - 2

- (3) Testing is conducted by the developer of the software and (for large projects) an independent test group
- (4) Testing and debugging are different activities, but debugging must be accommodated in any testing strategy

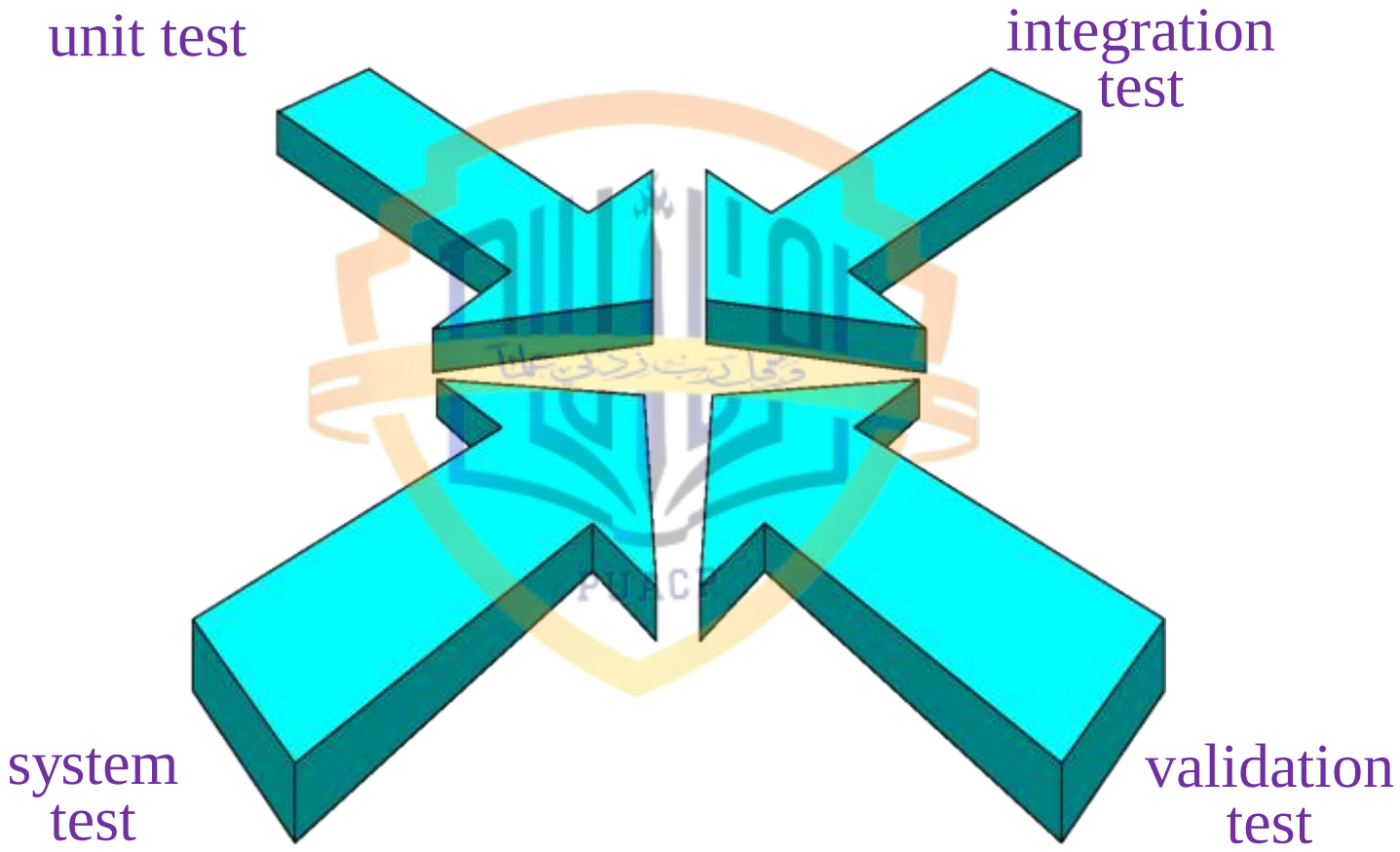
- A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high-level tests that validate major system functions against customer requirements

- A strategy must provide guidance for the practitioner and a set of milestones for the manager
- Because the steps of the test strategy occur at a time when deadline pressure begins to rise, progress must be measurable and problems must surface as early as possible

Testing Strategy - 1

- Unit testing
- Integration testing
- Validation testing
- System testing

Testing Strategy



Testing Strategy - 2

- Unit testing concentrates on each unit (i.e., component) of the software as implemented in source code
- Integration testing is next, where the focus is on design and the construction of the software architecture

Testing Strategy - 3

- Next is the validation testing, where requirements established as part of software requirements analysis are validated against the software that has been constructed
- Finally, we have system testing, where the software and other system elements are tested as a whole

Testing Strategy: Unit Testing

- Unit testing makes heavy use of white-box testing techniques, exercising specific paths in a module's control structure to ensure complete coverage and maximum error detection

Testing Strategy: Integration Testing

- Next, components must be assembled or integrated to form the complete software package
- Integration testing addresses the issues associated with the dual problems of verification and program construction
- BB test case design techniques are the most prevalent during integration

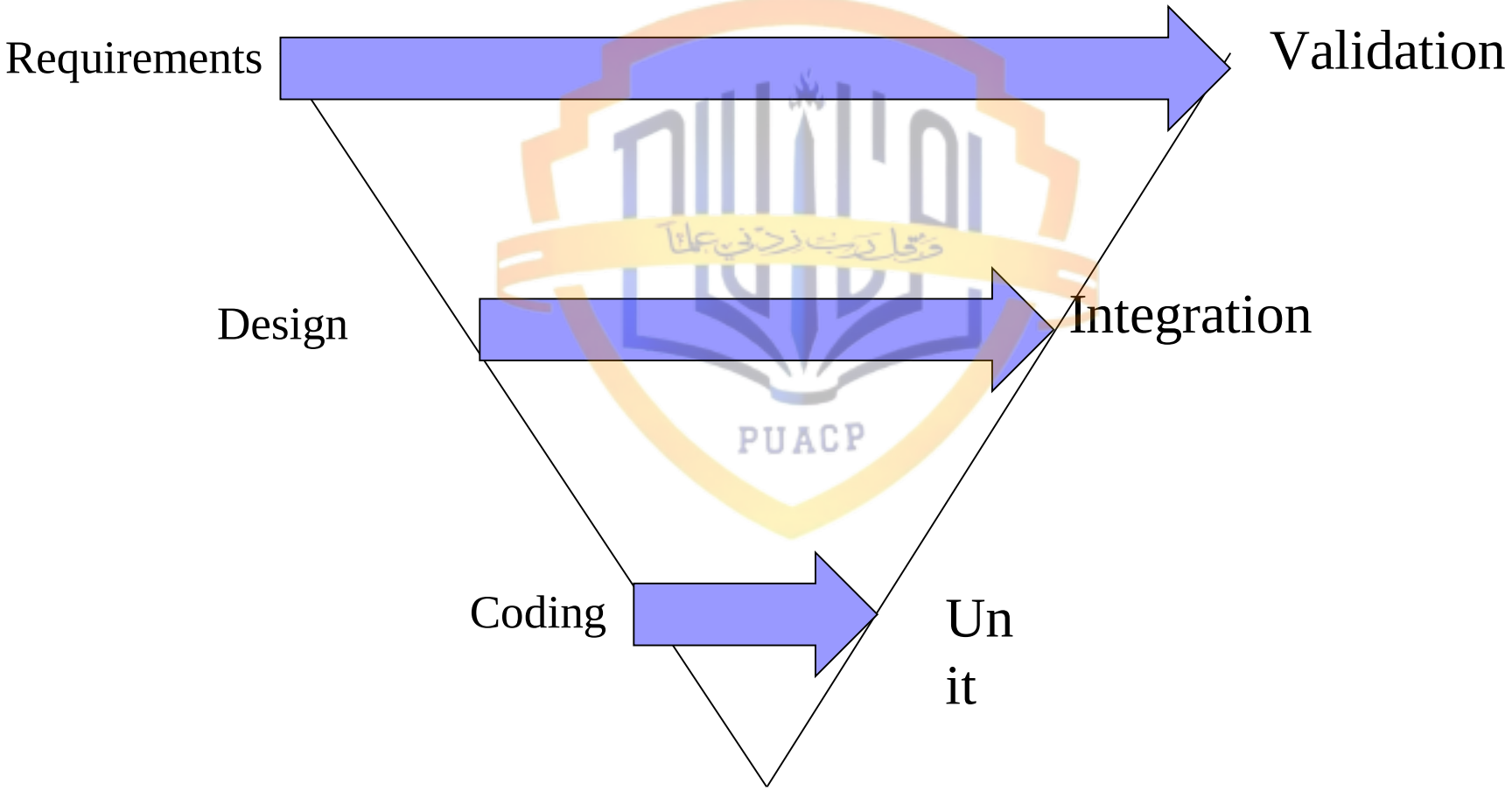
Testing Strategy: Validation Testing

- After software has been integrated, a set of high-order tests are conducted
- Validation criteria (established during requirements analysis) must be tested
- Validation testing provides final assurance that software meets all functional, behavioral, and performance requirements
- BBT techniques are used exclusively here

Testing Strategy: System Testing

- System testing is the last high-order testing and it falls outside the boundaries of software engineering and into the broader context of computer system engineering
- System testing verifies that all elements (hardware, people, databases) mesh properly and that overall system function/performance is achieved

The “V” Model of Software Testing



Strategic Issues - 1

- Specify product requirements in a quantifiable manner long before testing commences
  - NFRs
- State testing objectives explicitly
- Understand the users of the software and develop a profile for each user category
  - Actors/users of use-cases

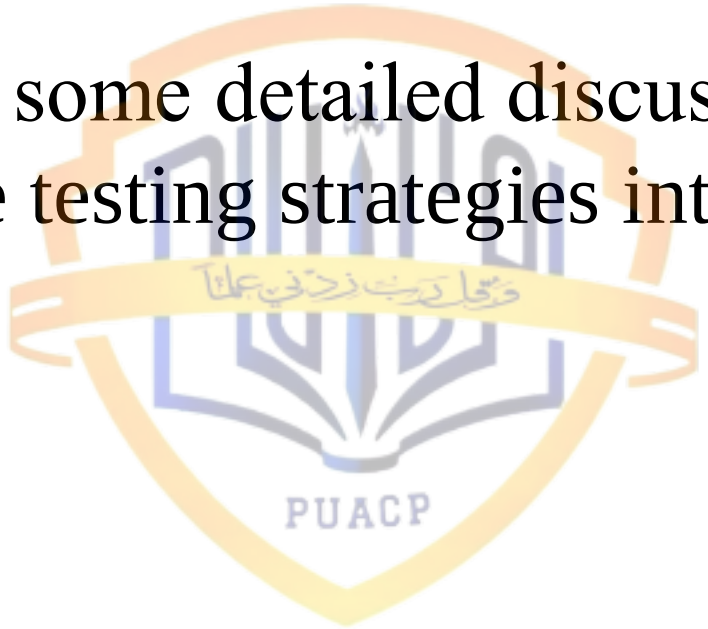
Strategic Issues - 2

- Develop a testing plan that emphasizes “rapid cycle testing”
- Build “robust” software that is designed to test itself
  - Software should be capable of diagnosing certain classes of errors
- Use effective formal technical reviews as a filter prior to testing

Strategic Issues - 3

- Conduct formal technical reviews to assess the test strategy and test cases themselves
- Develop a continuous improvement approach for the testing process

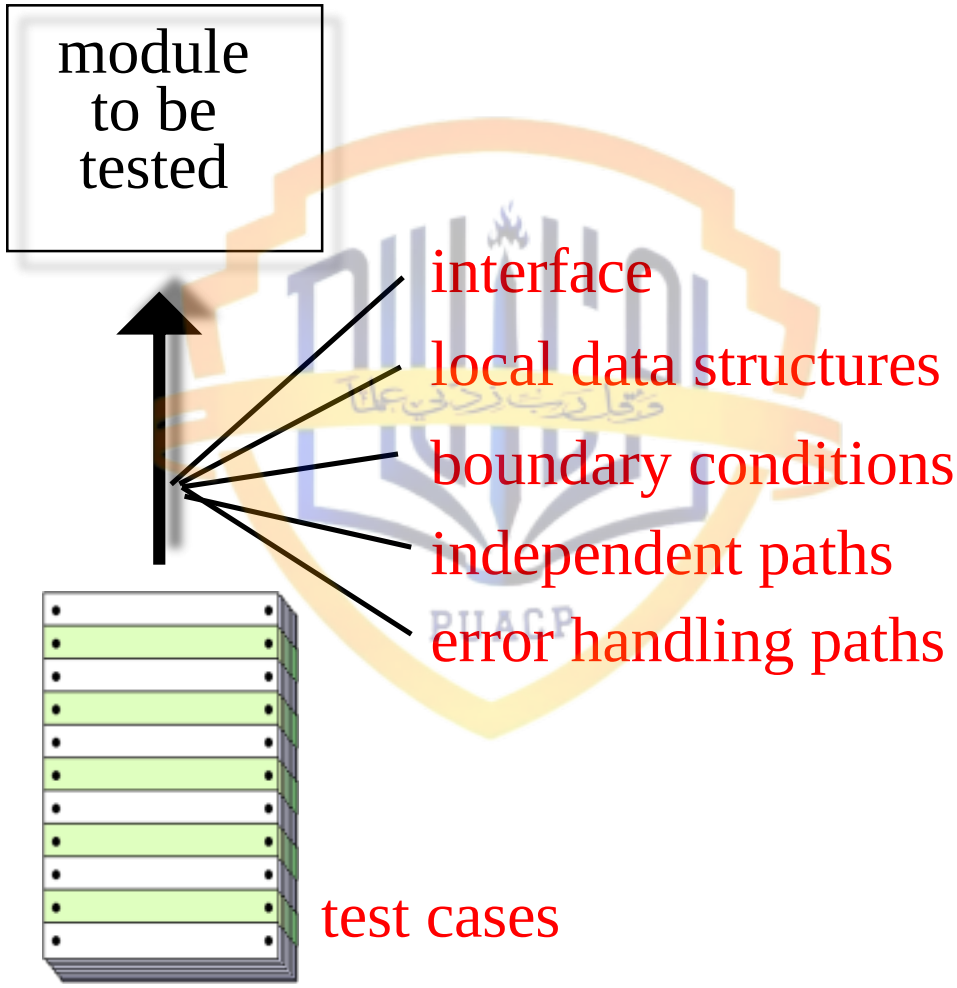
- Let’s have some detailed discussions on each of the testing strategies introduced



Unit Testing - 1

- Unit testing focuses verification effort on the smallest unit of software design – the software component or module
- The relative complexity of tests and uncovered errors is limited by the constrained scope established for unit testing
- Different units can be tested in parallel

Unit Testing



Unit Testing - 2

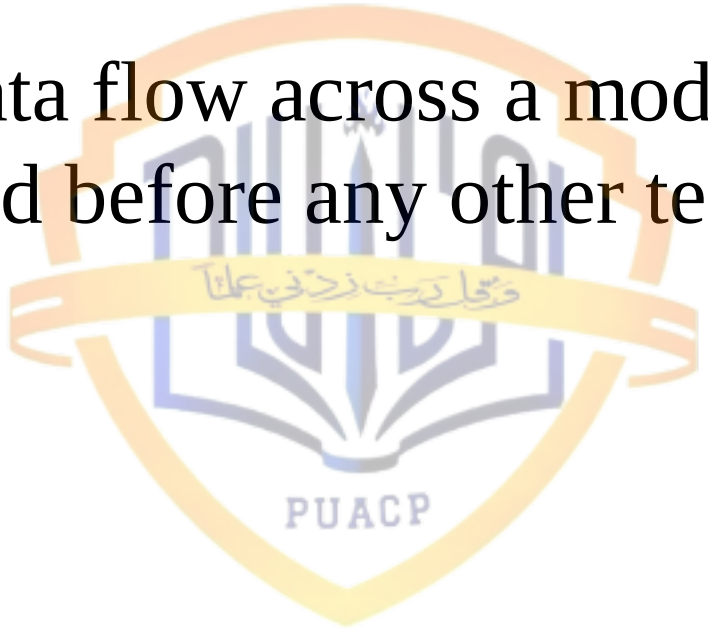
- The module interface is tested to ensure that information properly flows into and out of the program unit under test
- The local data structure is examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithm's execution

Unit Testing - 3

- Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restrict processing
- All independent paths through the control structure are exercised to ensure that all statements in a module have been executed
- All error handling paths are tested

Unit Testing - 4

- Tests of data flow across a module interface are required before any other test is initiated



Unit Testing: Common Errors Found

- Among the more common errors in computation are
  - Misunderstood or incorrect arithmetic precedence
  - Mixed mode operations
  - Incorrect initialization
  - Precision inaccuracy
  - Incorrect symbolic representation of expression

Unit Testing: Test Cases

- Test cases should uncover errors such as
  - Comparison of different data types
  - Incorrect logical operators or precedence
  - Expectation of equality when precision error makes equality unlikely
  - Incorrect comparison of variables
  - Improper or non-existent loop termination
  - Failure to exit when divergent iteration is encountered
  - Improperly modified loop variables

Unit Testing: Error Handling

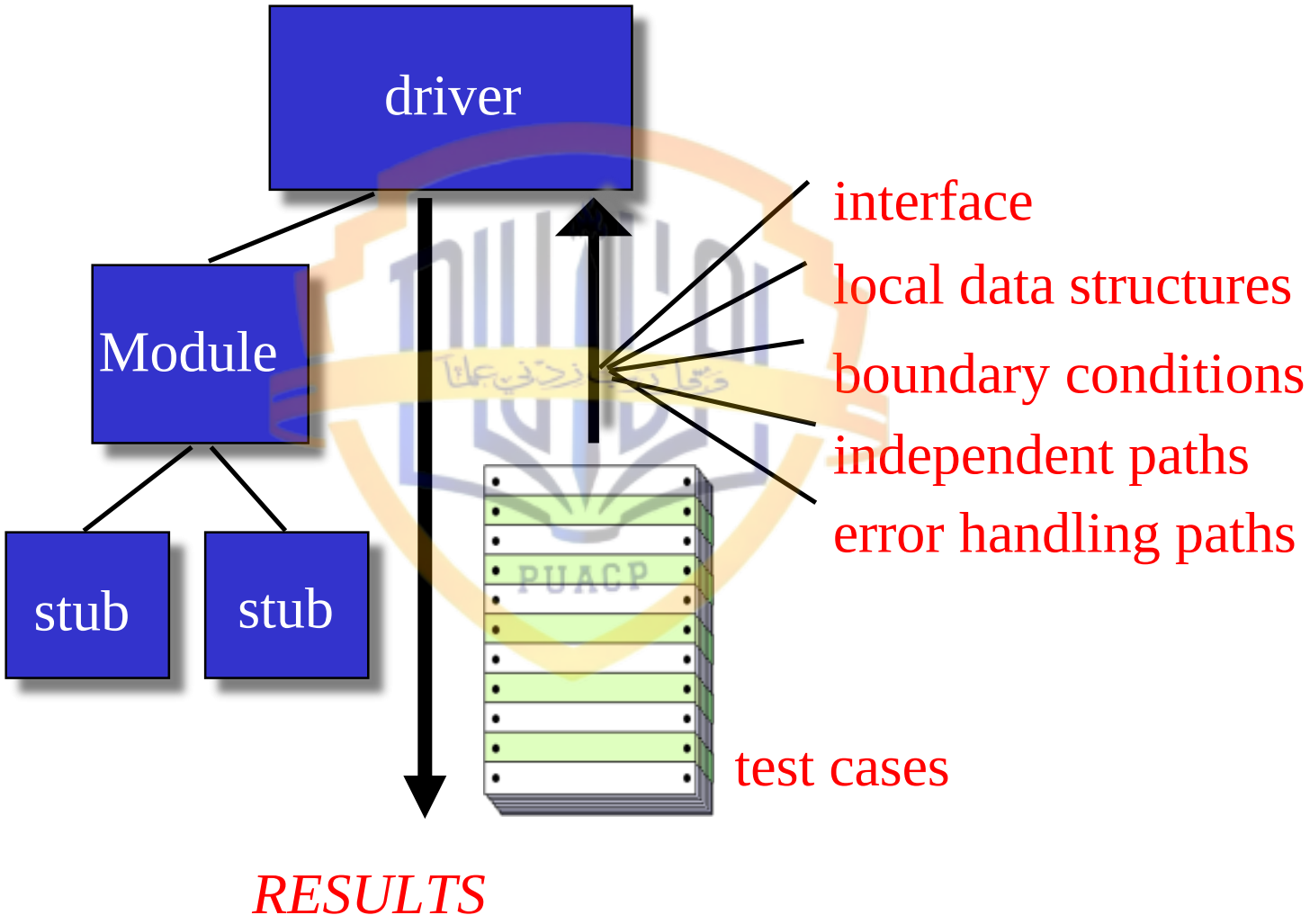
- Among the potential errors that should be tested when error handling is evaluated are
  - Error description is unintelligible
  - Error noted does not correspond to error encountered
  - Error condition causes system intervention prior to error handling
  - Exception-condition processing is incorrect
  - Error description does not provide enough information to assist in the location of the cause of the error

Unit Testing: Boundary Testing

- Boundary testing is the last and probably the most important task of unit testing step
- Software often fails at its boundaries
- Test cases that exercise data structure, control flow, and data values just below, at, and just above maxima and minima are very likely to uncover errors

- Unit testing is simplified when a component with high cohesion is designed
- When only one function is addressed by a component, the number of test cases is reduced and errors can be more easily predicted and uncovered
- Let’s see a unit test environment

Unit Test Environment



Integration Testing - 1

- "If they all work individually, why do you doubt that they'll work when we put them together?"
- The problem, of course, is "putting them together" – interfacing

Integration Testing - 2

- Data can be lost across an interface
- One module can have an inadvertent, adverse affect on another
- Sub-functions, when combined, may not produce the desired major function
- Individually acceptable imprecision may be magnified to unacceptable levels
- Global data structures can present problems

Software Integration

Integration Testing - 3

- Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing
- Individual components have already been unit tested and the structure is dictated by design

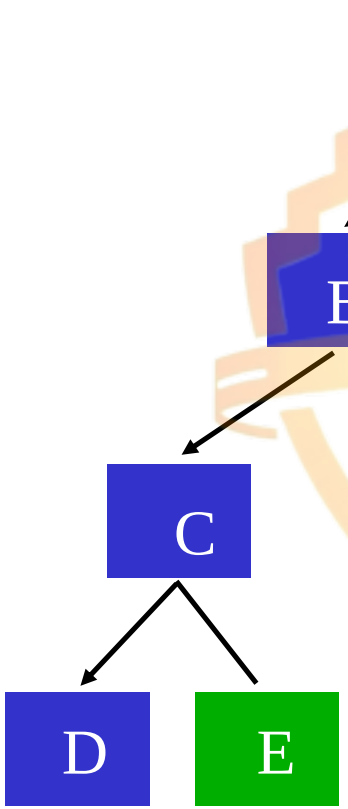
Integration Testing - 4

- There is tendency, sometimes, to attempt non-incremental integration – a sort of “big bang” approach
- All components are combined in advance
- The entire software is tested as a whole
- And chaos usually results!
- We should rather use incremental integration

Integration Testing Approaches

- Top-down integration
- Bottom-up integration
- Sandwich testing and integration

Top Down Integration



top module is tested with stubs

stubs are replaced one at a time, "depth first"

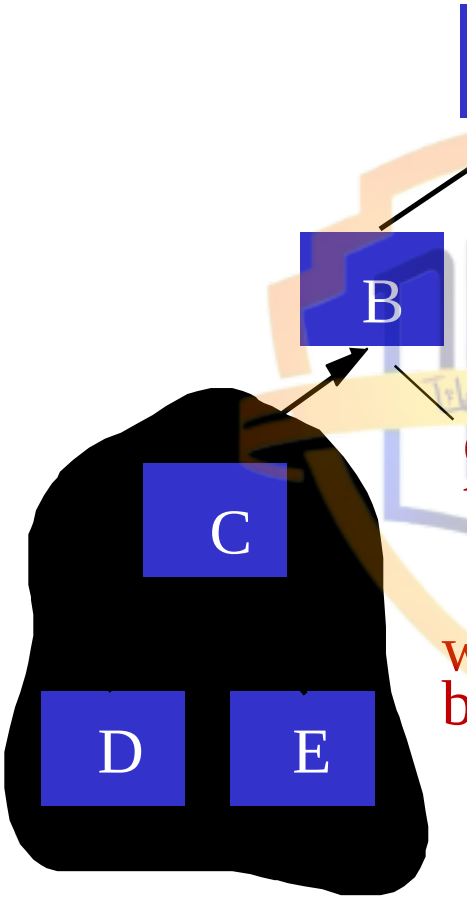
as new modules are integrated, some subset of tests is re-run

Following slide to be inserted

Bottom-Up Integration

40

Bottom-Up Integration



drivers are replaced one at a time, "depth first"

worker modules are grouped into builds and integrated

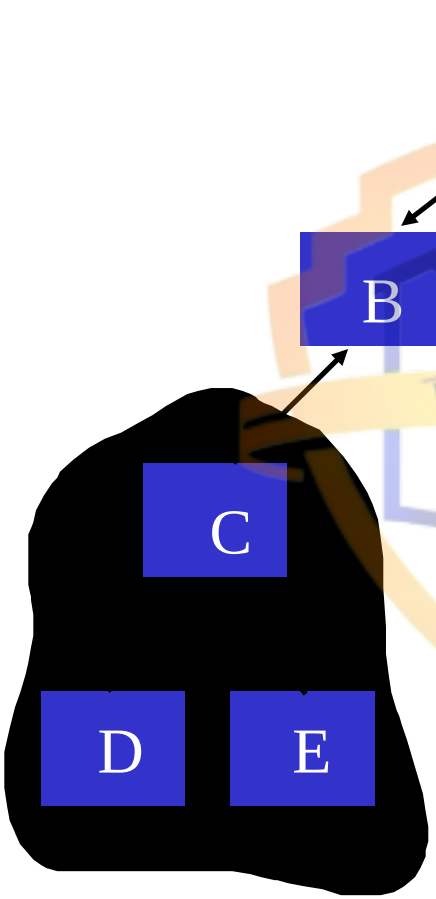
cluster

Following slide to be inserted

Sandwich Testing

42

Sandwich Testing and Integration



Top modules are tested with stubs

Worker modules are grouped into builds and integrated

cluster

- Regression testing

Summary

- Today we've discussed different testing strategies

References

- Software Engineering: A Practitioner's Approach by Roger S. Pressman (Chapter 18.1 – 18.4)